Accelerating AES-128

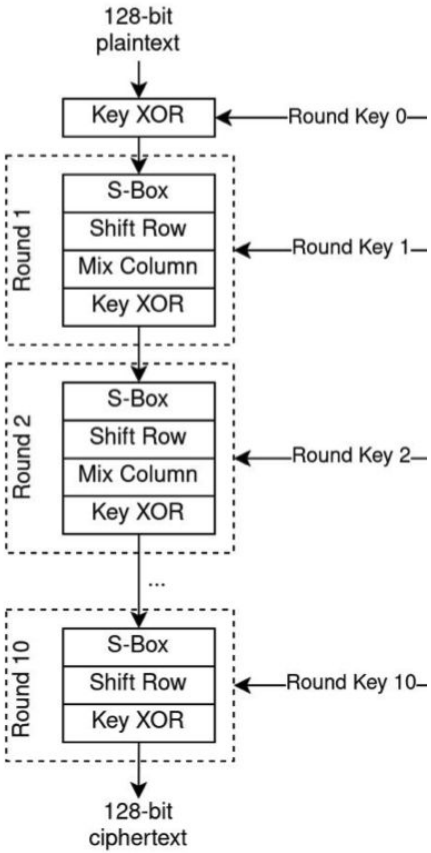
Michael Benney (z5478530)
Daniel Craig (z5417681)
Aaron Nyholm (z5316510)
Xavier Herbert (z5478413)

Problem outline and aims

- Encryption is vital to modern computing and required for a vase number of programs.
- Most encryption algorithms are heavily sequential when run on the CPU.
- AES is widely used for encryption across the internet and in storage.
- Accelerating AES on hardware provides benefit of offloading as well as increased parallelisation improving speed (particularly in ECB mode).

Aims:

- Implement AES in hardware to allow for encryption offloading
- Utilise parallelism of AES to provide encryption acceleration
- Reduce bottlenecks experienced with offloading to provide a seamless acceleration
- Achieve above goals without compromising security by exposing encryption key



Algorithm to be accelerated

AES-128 is an encryption algorithm that encrypts a fixed block of 128 bits.

AES-128 has 10 rounds where a range of operations including substitution, transposition, matrix multiplication and bitwise.

ECB mode is the simplest operation mode for AES as each fixed block of data is encrypted independently allowing for parallelism and pipelining.

In our stretch goals (milestone 4), we may choose to investigate accelerating AES-256, which keeps the 128 bit blocks of plaintext but instead generates 128 bit keys from a 256 bit secret key and has 14 total rounds.

AES pseudocode

```
#define NUM_ROUNDS 10
#define SIZE 128
#define SBOX_DIM = 16

D_TYPE round_keys[NUM_ROUNDS+1][SIZE] = { ... } // round keys for encryption
uint8_t rjindael_sbox[SBOX_DIM][SBOX_DIM] = { ... } // instantiated forward Rjindael's Sbox for sbox funct
void aes(D_TYPE plaintext[SIZE], D_TYPE ciphertext[SIZE]) {

    // key expansion
    key_expansion(round_keys);

    // define buffers
    D_TYPE key_xor_buf[SIZE], sbox_buf[SIZE], sr_buf[SIZE], mc_buf[SIZE];

    // Round 0
    key_xor(plaintext, round_keys[0], key_xor_buf);

    // Rounds 1 - 10
    for (int i = 1; i <= NUM_ROUNDS; i++) {
        // SBOX sub
        sbox(key_xor_buf, rjindael_sbox, sbox_buf);
        // shift row
        sr(sbox_buf, sr_buf);
        // if Round 1-9, Mix Cols, Else key_xor
        if (i < NUM_ROUNDS) {
            mc(sr_buf, mc_buf);
            key_xor(mc_buf, round_keys[i], key_xor_buf);
        } else {
            key_xor(sr_buf, round_keys[i], key_xor_buf);
        }
    }

    // output is in final buffer
    ciphertext = key_xor_buf;
}
```

Initial Investigation

AES is a round oriented sequential algorithm. Hence, we can pipeline multiple executions of AES-128 ECB through each of the 10 rounds at once.

5 unique stages:

- Key expansion - can store all round keys for ECB mode to aid with pipelining of rounds.
- Add Round Key - Quick XOR operation - minimal hardware
- Sub Bytes - lookup table operation - need sufficient hardware for 16 simultaneous byte lookups per SUB bytes stage - NOT enough BRAM will use LUTs, need roughly 5K 4 input LUTS for 14 instances
- Shift Rows - can be implemented as wires - need to ensure implemented as such from HLS - will be fast step
- Mix Columns - Matrix Multiplication - Will split into multiple stages to better match timing for other steps - Use Lookup tables for Multiplication (small fixed multipliers) - need 5K 4 input LUTS for 14 rounds as mul by 3 and bit shift for mul by 2

Kv260 board has 100k+ LUTs so complete unroll should be possible

HLS Constructs

Our initial investigation has turned up multiple HLS pragmas and constructs that will aid in optimising our design.

#pragma HLS dataflow - pipelines the AES rounds to ensure multiple instances of AES-128 can execute in parallel. Constructs separate pipeline stages from both functions and loops.

#pragma HLS array_partition ... - Allows us to store SBOX in FFs to provide parallel access to each stage as well as store every round key in such a way that each round can access all bits for key_xor at once

#pragma HLS pipeline/unroll - Allows us to parallelise some code execution, most important for the matrix multiplication of mix_cols function

Some other useful HLS constructs which we may find an application for:

- static thread_local D_TYPE : instantiate variables that are local to each instance of a function, most useful for counters
- hls::stream<D_TYPE,bit_size> / hls::stream_of_blocks<D_TYPE[bit_size], num_blocks> : instantiate a FIFO buffer that streams either single bits or chunks between pipeline blocks
- hls::task : apply in front of function calls to run them in parallel

Project Plan

Milestone 1 (Due by end of Week 6): Generate HLS code which implements an AES-128 algorithm with ECB. In parallel, generate a C testbench, and verify the functionality of the HLS code.

Milestone 2 (Due by end of Week 7): Connect the HLS core to the ARM processor's host program. Develop a means to pass plaintext to and receive ciphertext from the HLS core. Profile the performance and identify areas for improvement.

Milestone 3 (Due by start of Week 9): Working in parallel, optimise both HLS code and host program to alleviate bottlenecks.

Milestone 4 (Due by end of Week 10): Investigate accelerating AES-256 or AES in CBC mode (key is xored with previous round's cipher text. Profile optimised design and compare performance with running the algorithm on the ARM processor (purely host) and running AES on our laptops. Consider the security implications of our optimisations.

Risks and Team Roles

Risks and Contingencies: The primary risk of undertaking this project is the short turnover time of development. In order to mitigate this, we remain flexible with our final milestone, wherein if we have fallen behind it suffices to accelerate AES-128 in only ECB mode. Moreover, if any one area of development falls behind, our non-rigid team roles allows us to allocate manpower to wherever is needed.

Team Roles: Our team roles remain flexible as the project progresses, however our initial duties are as follows:

Aaron: Develop the host program and provide a means by which to feed in plaintext and extract cipher text from the kernel.

Daniel: Create a C testbench with which to test the HLS code and profile performance of different AES implementations.

Michael: Develop HLS implementation of AES-128 and work to optimise the storage elements of inputs, S-boxes and outputs.

Xavier: Develop HLS implementation of AES-128 and work to optimise the code structure through pipelining and unrolling.